

Rapport - système de simulation fibre optique ITU-T G.652

Général

AI and Machine Learning for Interferometric Optical Fiber Sensors: Simulation and Predictive Analysis of Temperature and Strain

1 / 1 projets

Les capteurs interférométriques à fibre optique permettent de mesurer température ou contrainte via des spectres optiques sensibles aux variations physiques.

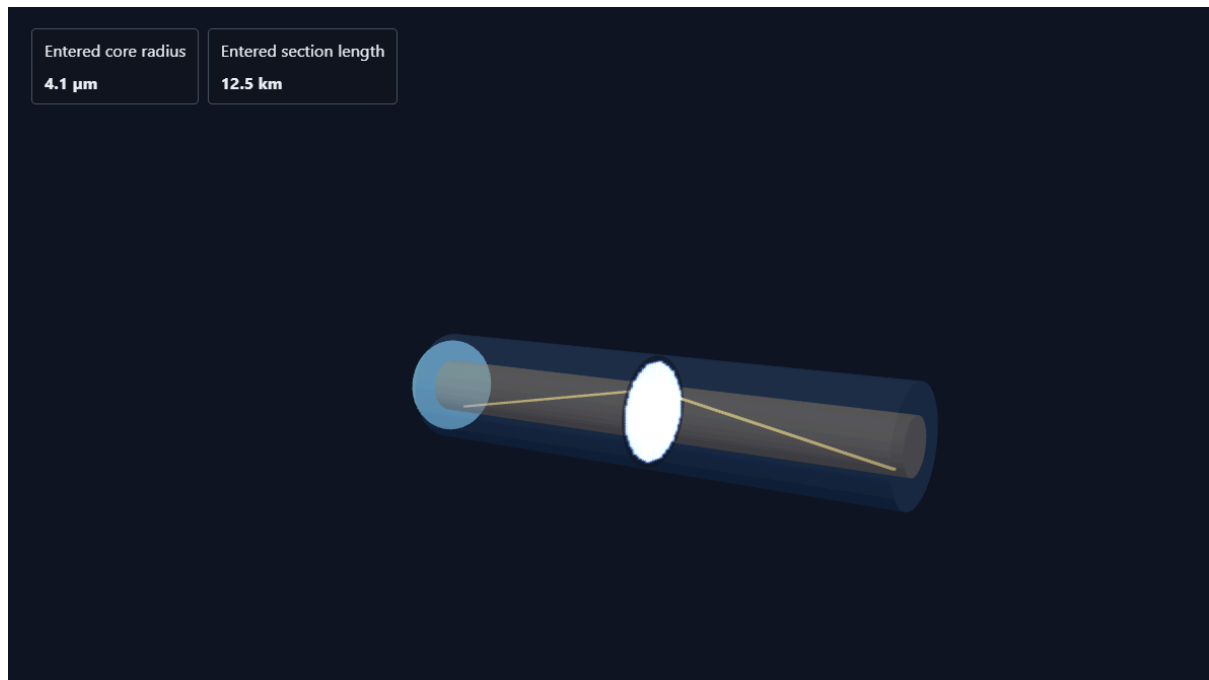
Données / dataset : dataset simulé de spectres interférométriques
Tâches principales : comprendre Fabry-Perot/Mach-Zehnder/Michelson/Sagnac, générer les données, entraîner régression linéaire/Random Forest/SVR/réseau léger, comparer MAE/RMSE/R²
Compétences : signaux simulés ; Python ; machine learning ; régression ; visualisation ; interprétation de résultats
Outils / stack : Python, Jupyter, NumPy, Pandas, scikit-learn, Matplotlib, Plotly, Streamlit/Dash
Livrables : rapport technique ; dataset ; notebook Python ; modèle IA ; dashboard Streamlit/Dash ; présentation finale
Modalités : Groupe de moins de 4 étudiants

Déposant : Mohamed AAZI

#IA #Machine Learning #Capteurs #Fibre optique

Auteur
Mohamed AAZI
maazi@ece.fr

Objectif -> comprendre comment le câble fonctionne (avec chap 7&8) ensuite se concentrer sur le projet avec ia



-> tuyau en verre très fin (la fibre optique). Dedans, on envoie de la lumière pour transporter de l'information (internet, téléphone...).

1. changes des chiffres (longueur du tuyau, combien de lumière se perd...)
2. ordinateur recalcule
3. on voit le résultat en 3D et en courbes

Objectifs principaux :

1. Implémenter des modèles physiques déterministes (guidage, modes, atténuation, dispersion).

2. Les composer en une simulation unifiée (« Level 1 »).
3. Les exposer via API REST + interface web (3D, graphiques, standards).
4. Garantir la qualité (tests, typage, contrats OpenAPI).

Comment le logiciel est rangé ?

Comme une calculatrice : moi → écran → serveur → formules → résultat → écran.
 apps/web (React) → apps/api (FastAPI) →
 packages/physics_core (fibre_sim) (**JSON / OpenAPI**)

composant	rôle	techno
physics_core	calculs physiques	python 3.12
apps/api	api rest	fastapi
apps/web	ui	react, vite, typescript
shared_chemas	contrat parta	openapi types ts
docker/makefile	lancement	make dev -> http://localhost:5173

Précis = Artem

- Monorepo : frontend, API, moteur physique, schémas partagés, tests, scripts et déploiement dans un seul dépôt.
- Frontend React : React, TypeScript et Vite pour l'application interactive dans le navigateur.
- Backend FastAPI : API Python versionnée qui valide les requêtes et exécute tous les modèles numériques.
- Paquet physique installable : `fibre-sim` isole les calculs physiques et normatifs de la couche web.
- Dépendances verrouillées : Python via uv ; frontend avec lockfile npm versionné.
- Exécution conteneurisée : images Docker non-root et profils Compose pour lancer frontend et API de façon reproductible.
- Santé : l'API expose un endpoint de santé ; le frontend signale la disponibilité du backend.
- Automatisation qualité : la CI lance formatage, lint, typage, tests, contrôles de contrats et builds de conteneurs.

Flchiers repères :

Appli qui tourne `apps/web/src/App.tsx`
 Formules Level 1 `packages/physics\_core/src/fibre\_sim/level1/calculations.py`
 API `apps/api/app/main.py`
 Scène 3D `apps/web/src/FibreGeometryView.tsx`

Que se passe-t-il quand tu bouges un curseur ?

1. changes un nombre (paramètre -> fibre source, section & sampling)
2. ordi attend ($\frac{1}{4}$ de seconde) pour ne pas recalculer à chaque lettre (debounce 250 ms puis `POST /api/v1/simulations/preview`)
3. Il envoie les chiffres au serveur
4. serveur calcule. (valida pydantic -> calculate_level1_simulation)
5. écran se maj : fibre en 3D, graphiques, notes.

Si tu modifies souvent, ça peut aller jusqu'à environ 4X par seconde. Ce n'est pas une vidéo qui tourne toute seule : ça recalcule qd tu changes quelque chose

Précis = Artem

- Entrées scientifiques strictes : modèles Pydantic fermés ; rejet des valeurs manquantes, en trop, non finies ou physiquement invalides.
- Unités explicites : noms de champs et schémas API indiquent l'unité (ou le caractère sans dimension).
- Manifestes de modèle versionnés : chaque résultat identifie modèle, version, hypothèses, limites de validité et exclusions.
- Avertissements structurés : un sous-modèle peut renvoyer une valeur indisponible + un warning précis sans masquer le reste.
- Erreurs API traçables : enveloppe d'erreur stable avec identifiants de trace.
- Contrats client générés : OpenAPI déterministe + types TypeScript pour aligner frontend et backend.
- Fondation de domaine : fibres, sections de câble, épissures, connecteurs, séries échantillonnées, coupes de champ — pour les extensions futures.

La physique : ce que le programme calcule

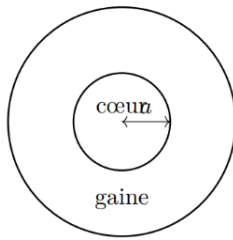
idée général -> La lumière dans la fibre, c'est un peu comme une balle qui rebondit dans un tunnel

- si elle tape "assez à plat", elle reste dedans (**guidage**)
- en avançant, elle perd un peu de force (**atténuation**)
- un flash lumineux peut s'étaler en voyageant (**dispersion**)
- on vérifie aussi si les chiffres sont OK pour une vraie fibre G.652 (**norme**)

guidage → forme de la lumière → pertes → temps de trajet → élargissement du flash →
(option) contrôles norme

« Est-ce que la lumière reste dans le tuyau ? »

La fibre a un cœur (dedans) et une gaine (autour).



(a) section transverse

Le cœur est un peu « plus lent pour la lumière » (indice plus élevé).

Résultat : la lumière peut rebondir sur le bord et rester captive (comme un miroir intérieur)

notre programme répond à des questions du type :

- À quel angle maximum la lumière est encore « piégée » ?
- La fibre est-elle monomode (un seul « chemin » propre) ou multimode (plusieurs chemins) ?

Modèle idéal : fibre circulaire à saut d'indice (chap. 8 §8.1). Le chap. 7 pose d'abord la réflexion totale et les modes (guides plans, §7.2).

Précis = Artem

- **Ouverture numérique (NA)** à partir des indices cœur / gaine.
- Angle critique cœur–gaine (depuis la normale à l'interface).
- Angle d'acceptance dans l'air quand le modèle arcsinus est valide.
- Différence relative d'indice selon la convention documentée.
- Fréquence normalisée V avec conversions explicites rayon / longueur d'onde.
- Classification de mode : monomode / multimode au seuil $V = 2.405$.
- Estimation du nombre de modes : asymptotique $V^2/2$ seulement pour $V \geq 10$, sinon warning.
- Agrégation du guidage : valeurs, warnings et métadonnées en un résultat déterministe.

Endpoint dédié : `POST /api/v1/guidance/calculate`.

« à quoi ressemble la tache de lumière ? »

Dans une fibre monomode, la lumière n'est pas un trait parfait : c'est une tache ronde, plus brillante au centre, plus faible sur les bords comme une lampe torche floue. Le programme dessine cette tache sur une grille (comme des pixels) pour la montrer en 3D / en graphique.

Précis = Artem

- Rayon de champ (Marcuse) : estimation gaussienne du mode-field radius dans la plage V déclarée.
- Profil LP01 gaussien : échantillonnage scalaire à symétrie circulaire sur une grille carrée impaire.

- Champ et intensité : matrices normalisées de champ et d'intensité (champ au carré) avec coordonnées exactes.
- Conventions : distinction rayon de champ $\backslash(1/e\backslash)$, rayon d'intensité $\backslash(1/e^2\backslash)$, et MFD.

« la lumière s'affaiblit en route »

+ le câble est long, plus la lumière perd de la force (absorption, diffusion dans le verre)

ex -> tu perds 0,2 dB par km et que le câble fait 10 km, tu perds 2 dB au total.

En dB, on a les pertes (c'est plus simple que de multiplier des pourcentages)

Précis = Artem

- Atténuation constante : **coefficient × longueur de section.**
- Puissance optique en **dBm** (sans convertir incorrectement des grandeurs logarithmiques).
- Échantillons puissance-distance pour les courbes et les indicateurs spatiaux 3D.
- Retard de groupe : longueur, indice de groupe, vitesse de la lumière dans le vide.
- Dispersion chromatique accumulée (signée) sur la section.
- Élargissement d'impulsion : combinaison en quadrature largeur d'entrée + élargissement de dispersion (FWHM gaussien, 1er ordre).
- Pertes de macrocourbure fournies par l'utilisateur : jusqu'à 32 courbures ordonnées (position, rayon, angle, perte passive explicite).
- Puissance après courbures : pertes de bend appliquées après l'atténuation droite, reportées séparément dans Level 1.

« le flash s'étale »

1. Retard de groupe combien de temps le « paquet » de lumière met pour arriver.

Comme le temps qu'un train met pour parcourir la voie.

2. Élargissement un flash court au départ peut devenir un flash plus large à l'arrivée, parce que les couleurs (longueurs d'onde) ne voyagent pas exactement à la même vitesse.

Comme une course où tout le monde ne court pas à la même allure : le peloton s'étire.

Si le flash s'étale trop, les bits d'info se mélangent → on ne peut plus envoyer aussi vite.

« est-ce que ces chiffres sont raisonnables pour une vraie fibre ? »

Il existe une règle internationale pour les fibres télécom courantes (**G.652**).

Le programme peut vérifier : « ton atténuation / ta dispersion, c'est dans la zone autorisée ? »

Comme un contrôle technique : vert / rouge, mais un seul contrôle vert ≠ voiture parfaite.

Précis = Artem

- Preset sourcé : sépare les limites représentées de la norme et les défauts de simulation informatiques.

- Défauts conservateurs à la sélection du preset : 1550 nm, 0,275 dB/km, 17 ps/(nm·km).

- Catalogue de limites représentées : MFD, géométrie de gaine, cut-off, qualification macrobend, preuve mécanique, atténuation, dispersion, PMD (métadonnées).
- Enveloppe de dispersion G.652.D évaluée de 1260 à 1625 nm.
- Contrôle dispersion : coefficient fourni vs min/max inclusifs dépendant de λ .
- Contrôle atténuation : valeur fournie vs limites représentées (longueur d'onde, application câble).
- Applicabilité : contextes non supportés → « not applicable » plutôt qu'un faux « pass ».
- Provenance : édition, références sources, attributs représentés, limitations.
- Pas de fausse conformité : réussir les checks représentés ≠ certification complète.

level 1 -> tout coller en un seul résultat

Au lieu de lancer cinq calculatrices séparées, Level 1 dit :

« Donne-moi tous les réglages une fois → je te rends tout le bilan. »

Tu fournis : la fibre, la source lumineuse, la longueur du tronçon, la précision du dessin.

Tu reçois : guidage, tache de mode, pertes, retard, élargissement, (éventuellement) contrôles norme, plus des avertissements si un calcul optionnel n'est pas possible.

Entrée : `preset`, `fibre`, `source`, `section`, `sampling`.

Sortie : `guidance`, `mode\_profile`, `attenuation`, `group\_delay`, `pulse\_broadening`, `standards\_checks`, `warnings`, `model\_manifest`.

Chaque sous-modèle a un manifest (`model\_id`, version, hypothèses, limites). Aucune formule n'est réécrite dans la couche de composition.

Fonctionnalités Level 1 / API associées :

- Composition unifiée : guidage, champ de mode, atténuation, bends, retard, élargissement, checks norme, warnings.
- Frontières de paramètres : limites d'entrée, de validité de modèle, monomode idéal et G.652.D pour les champs concernés.
- **`POST /api/v1/simulations/preview` — résultat Level 1 synchrone complet.**
- **`POST /api/v1/simulations/sweep` — un paramètre choisi sur 2–200 échantillons linéaires.**
- Documentation interactive : OpenAPI versionné, Swagger UI, ReDoc.

API (le serveur)

L'écran parle au serveur par Internet local, avec des adresses du genre :

- « tu es vivant ? » → health

- « calcule tout » → preview

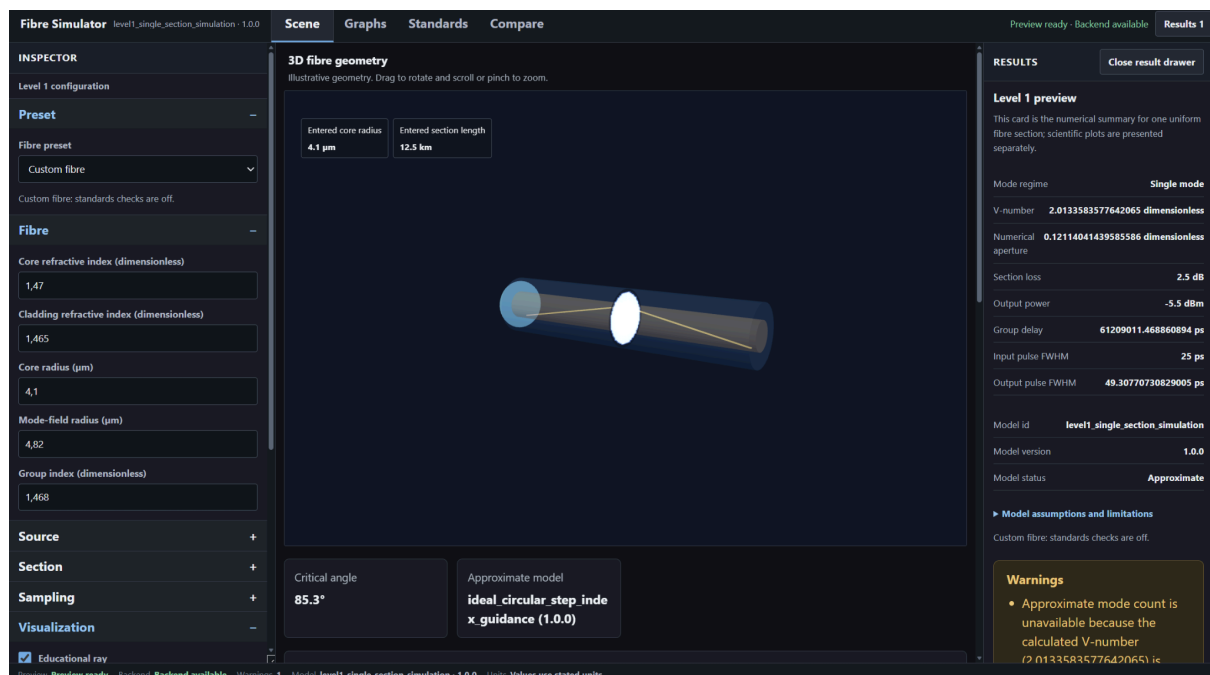
Si ça casse, le serveur renvoie un message d'erreur clair (pas juste « error »)

methode	chemin	role
GET	/api/v1/health	Santé
POST	/api/v1/guidance/calculate	Guidage seul
POST	/api/v1/simulations/preview	level 1

Erreurs `ErrorResponse` + `X-Trace-ID`. Calculs non finis → `CALCULATION\_ERROR` (422). Contrats OpenAPI synchronisés avec le frontend (`make contracts`)

Interface web

Quatre « pièces » dans l'appli :



Scene -> Le dessin 3D du câble et de la lumière

Graphs -> Les courbes (puissance, forme du flash..)

Standards -> Le feu vert / rouge de la norme

Compare -> Prévu pour comparer

à côté : les boutons et cases pour changer les paramètres.

c'est un éditeur scientifique

- Formulaire Level 1 groupé (fibre, source, section, contexte câble, sampling) avec unités explicites.

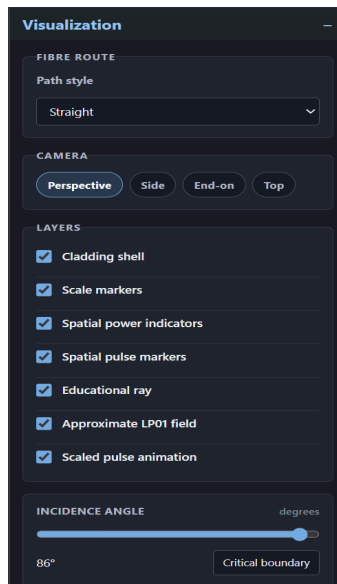
- Modes **Custom** et **G.652.D**

- Preview immédiat (debounce) sans bouton « Submit » obligatoire.

- Protection anti-résultats périmés (annulation + séquençage des requêtes).

- Guidage champ par champ (frontières, feedback jaune / rouge)

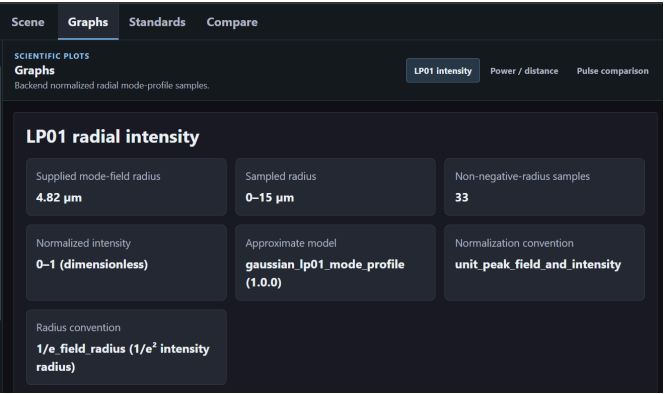
- Espaces de travail : Scene, Graphs, Standards, Compare, Sweep
- Tiroir de résultats : numériques, warnings, hypothèses, limites, identité du modèle, statut norme



- Géométrie cœur / gaine (cœur solide normalisé, gaine translucide illustrative).
- Routes d'affichage : droite, arc doux, S-bend.
- Caméra : orbit, zoom, presets nommés, état caméra personnalisé restaurable.
- Couche rayon éducatif : transmission / limite critique / réflexion totale (depuis le guidage backend).
- Couche champ LP01 : échantillons backend → champ de points d'intensité normalisée.
- Animation d'impulsion mise à l'échelle (enveloppe entrée → sortie sur une durée d'affichage fixe).
- Indicateurs spatiaux de puissance (fractions de distance physiques).
- Marqueurs d'extrémités d'impulsion (FWHM entrée / sortie).
- Marqueurs d'échelle (fractions de parcours ↔ longueur physique saisie).
- Couches indépendantes (gaine, rayon, mode, pulse, puissance, marqueurs...).
- Légende accessible (noms de routes, valeurs exactes).
- Rendu efficace : à la demande, ratio de pixels borné, repli si WebGL indisponible.

Graphiques scientifiques

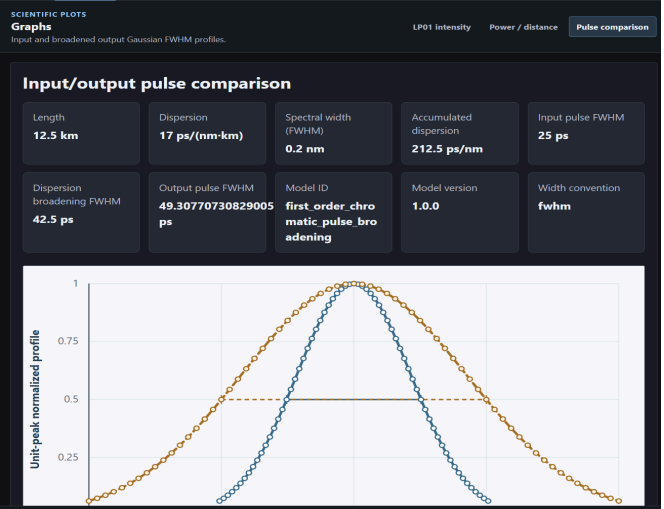
- Intensité radiale LP01 : SVG + tableau



- Puissance vs distance : SVG + tableau



- Comparaison d'impulsions : profils gaussiens normalisés (FWHM entrée / sortie)



- Validation stricte : données manquantes, périmées, malformées, longueur nulle, constantes → messages explicites.

REPRESENTED CHECKS
Standards
Custom fibre - standards checks are not applicable
Custom

Attenuation
! Not applicable

Select the G.652.D preset to compare attenuation with represented standard-cable limits.

Chromatic dispersion
! Not applicable

Select the G.652.D preset to compare the supplied coefficient with the represented wavelength envelope.

« on vérifie que ça ne se trompe pas »

avant de faire confiance à une calculatrice, on teste plein d'exemples
ici -> tests automatiques, des vérifications de style de code, et une machine (CI) qui rejoue tout ça à chaque changement.

Il ne simule pas (pour l'instant) : les nœuds / soudures, les courbes trop serrées, plusieurs tronçons à la suite, les capteurs de température, ni l'IA.

Conclusion

on a créé une appli qui calcul et montre comment marche une fibre télécom G.652.
Elle prend les idées des chap 7 et 8, les met dans du code, et les affiche pour qu'on voie ce que les formules font

dossier ->

- apps/api/

API FastAPI
- apps/web/

Frontend React
- packages/physics_core/

Moteur fibre_sim + docs + tests
- packages/shared_schemas/

OpenAPI + types TS
- scripts/

Export OpenAPI
- compose.yaml / Makefile

Docker & commandes

annexe = récap et reliage des formules au code

	En simple	formule	ref	code
D1	Loi du rebond / réfraction	$\sqrt{n_1 \sin \theta_1} = \sqrt{n_2 \sin \theta_2}$	Thèse (1.1) ; chap. 7	conceptuel
D2	Angle où ça « colle » au bord	$\theta_c = \arcsin(n_2/n_1)$	Chap. 8 §8.1A ; thèse (1.2)	<code>`critical_angle_deg`</code>

D3	Capacité à « attraper » la lumière	$\mathrm{NA} = \sqrt{n_1^2 - n_2^2}$	(8.1-3)	`numerical_aperture`
D4	Cône d'entrée dans l'air	$\theta_a = \arcsin(\mathrm{NA})$	(8.1-2)	`air_acceptance_angle_deg`
D5	Écart relatif des indices	$\Delta = (n_1 - n_2)/n_1$	Ex. 8.1-1	`relative_index_difference`
D6	Score mono / multi	$V = 2\pi a \mathrm{NA} / \lambda_0$	§8.1C ; thèse (1.3)	`v_number`
D7	Un seul mode ?	$V < 2.405$	§8.1C / Table 8.1-1	`classify_mode_regime`
D8	Combien de modes (approx.)	$M \approx V^2/2$	§8.1B $\approx \sqrt{4V^2/\pi^2}$	`approximate_mode_count`
D9	Forme de la tache	(E, I) gaussiens	§8.1C	`modes/calculations.py`
D10	Combien on perd par km	α dB/km	(8.3-1)	`attenuation`
D11	Puissance qui reste	$P(z)/P(0) = 10^{-\alpha z/10}$	(8.3-2)	`calculate_constant_attenuation`
D12	Perte totale du tronçon	$A = \alpha L$	§8.3A (dB additifs)	samz
D13	Temps d'arrivée du paquet	$t_g = n_g L/c$	Chap. 7 ; chap. 8 §8.1B/§8.3	`calculate_group_delay`
D14	Le flash s'étire	$\Delta T \propto D \Delta \lambda$	(8.3-6)/(8.3-7)	`calculate_chromatic_pulse_broadening`
D15	Largeur finale	$\sqrt{T_{in}^2 + \Delta T^2}$	(8.3-16)	same
D16	Contrôle norme	Table 2 G.652	ITU-T G.652 (08/2024)	`standards/*`

